Analyze Your First Packet with Wireshark

Background

Network traffic analysis is an important task for a security analyst. Network packets contain information about communication between devices, including source and destination IP addresses, protocols, ports, and transmitted data.

Wireshark is a network protocol analyzer that allows analysts to inspect captured network traffic through a graphical user interface (GUI). Display filters help narrow down large packet captures to find information relevant to a security investigation.

Activity Overview

In this lab, I used Wireshark to analyze a sample packet capture file (`.pcap`) containing traffic related to a user accessing a website.

The activity focused on exploring the Wireshark interface, inspecting packet details, filtering traffic by IP and MAC addresses, examining DNS queries and responses, and searching TCP packets for specific text.

Objective

- Explore a sample packet capture in Wireshark.
- Identify source and destination IP addresses.
- Examine Ethernet, IPv4, TCP, UDP, ICMP, and DNS information.
- Apply display filters to find relevant packets.
- Inspect packet properties, including frame length, IP header length, Time to Live (TTL), and TCP ports.
- Search TCP packet data for specific text.

Tools Used

- **Wireshark:** Used to open and analyze the packet capture.
- **Sample PCAP file:** Provided lab data containing network packets.
- **Windows virtual machine:** Used to run Wireshark in the lab environment.

Lab Environment

The analysis was performed in the Windows virtual machine provided by the course lab. The supplied sample packet capture was used rather than collecting live traffic.

Task 1: Explore Data with Wireshark

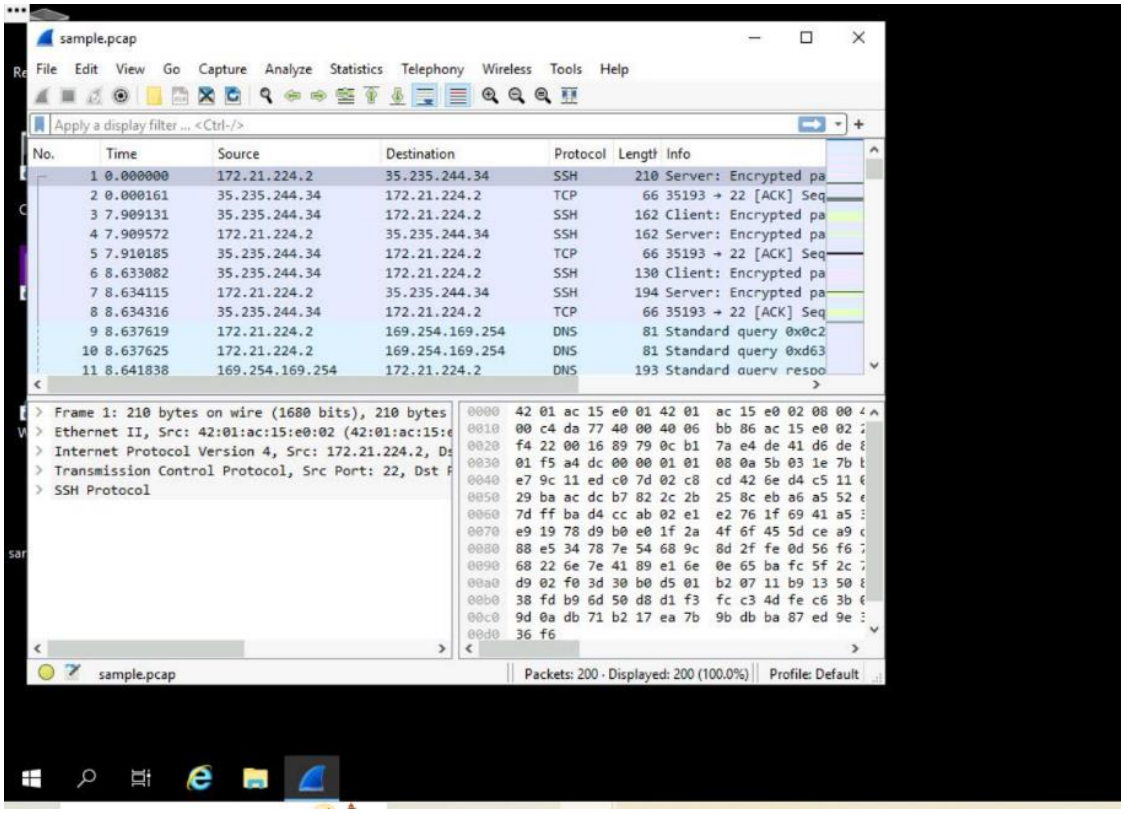
I opened the sample packet capture file in Wireshark and explored the main packet list.

The interface displayed packet numbers, timestamps, source addresses, destination addresses, protocols, packet lengths, and information about each packet.

I also reviewed the packet coloring rules, which help distinguish different types of network traffic.

Observation: The first packet whose Info field began with `Echo (ping) request` used the ICMP protocol.

Screenshot 1 — Wireshark Overview



Task 2: Apply an IP Address Filter and Inspect a Packet

I applied a display filter to identify packets associated with a specific IP address.

Display filter

```
ip.addr == 142.250.1.139
```

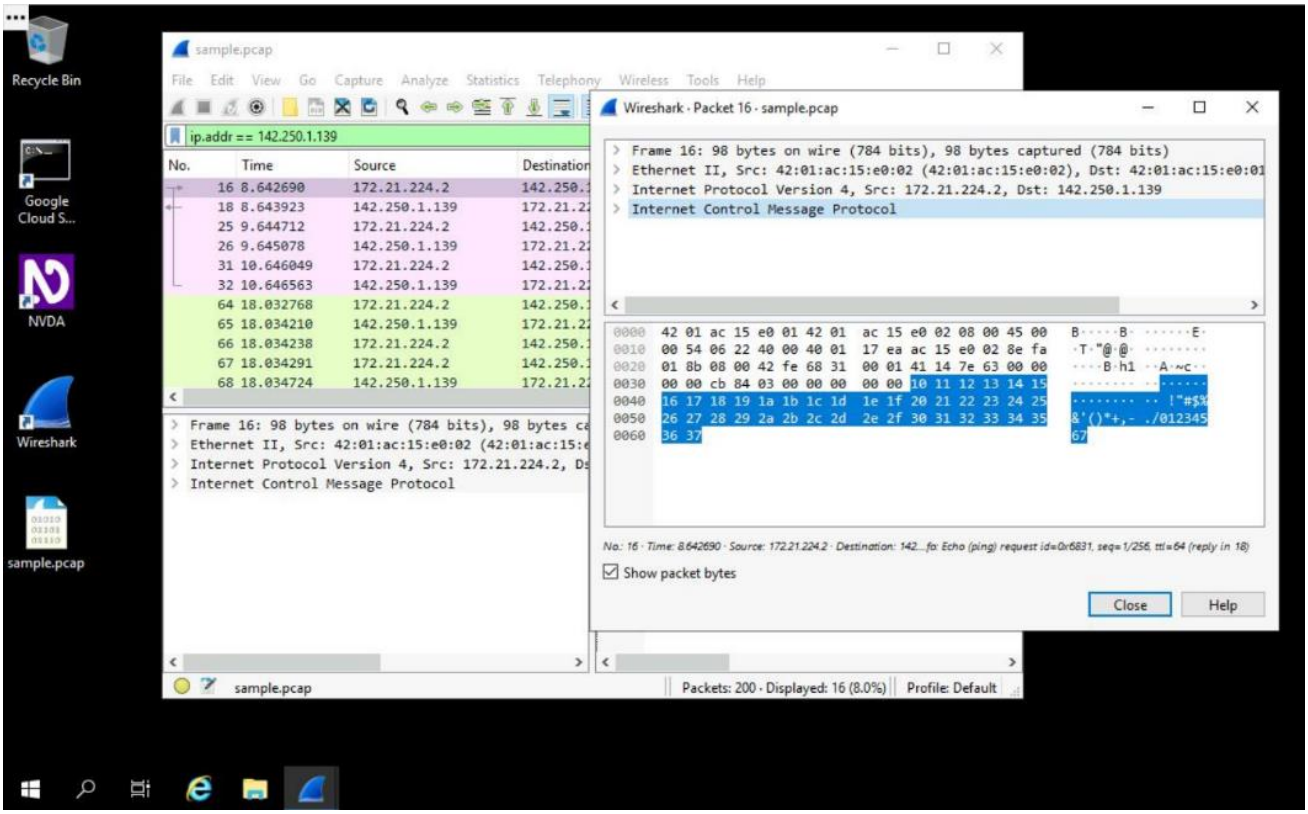
This filter displays packets where `142.250.1.139` is either the source or destination IP address.

I opened a TCP packet and inspected its protocol layers:

- **Frame:** Information about the captured packet, including its length and arrival time.
- **Ethernet II:** Source and destination MAC addresses.
- **Internet Protocol Version 4:** Source and destination IP addresses and the internal protocol.
- **Transmission Control Protocol:** Source and destination ports, sequence numbers, and TCP flags.

Finding: The TCP destination port was `80`, which is commonly used for HTTP traffic.

Screenshot 2 — IP Address Filter



Task 3: Filter Packets by Source, Destination, and MAC Address

I used separate display filters to examine traffic based on its source IP address, destination IP address, and Ethernet MAC address.

Source IP address

```
ip.src == 142.250.1.139
```

This filter displays packets originating from the specified IP address.

Destination IP address

```
ip.dst == 142.250.1.139
```

This filter displays packets sent to the specified IP address.

Ethernet MAC address

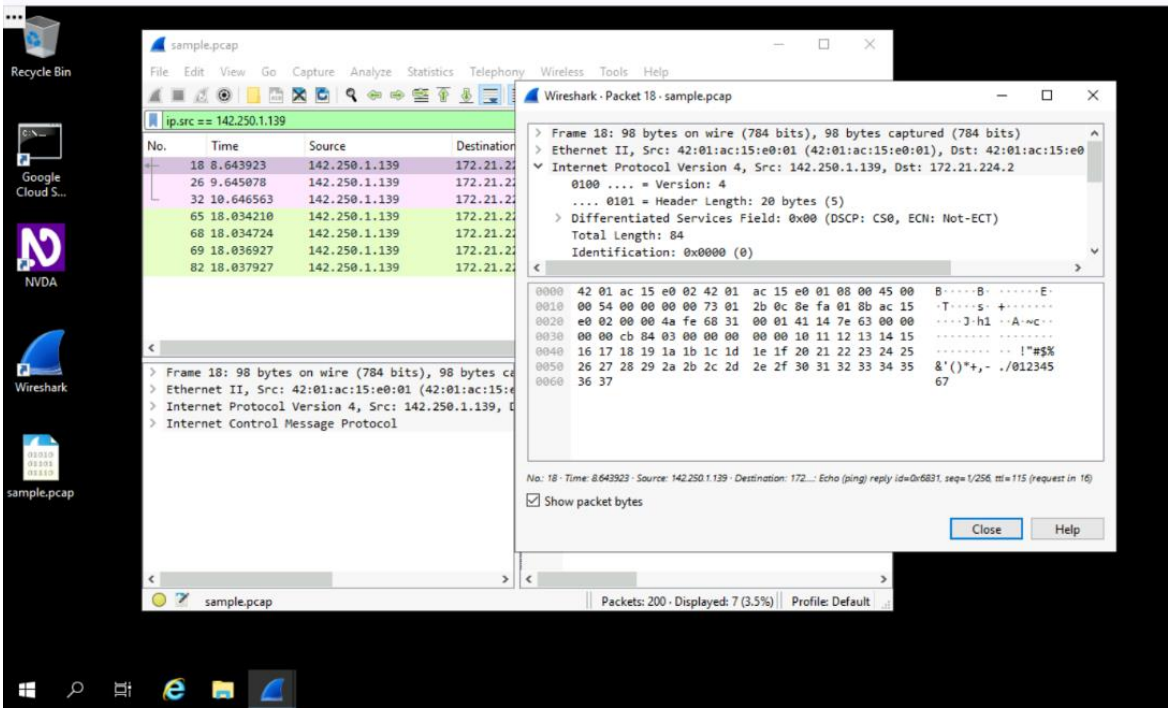
```
eth.addr == 42:01:ac:15:e0:02
```

This filter displays packets where the specified MAC address appears as either the source or destination address.

After opening a matching packet, I inspected the Ethernet II and IPv4 sections.

Finding: The internal protocol shown in the IPv4 section of the first matching packet was TCP.

Screenshot 3 — Source, Destination, and MAC Filtering



Task 4: Explore DNS Packets

Domain Name System (DNS) translates domain names into IP addresses. In this task, I filtered UDP traffic associated with port 53 to examine DNS queries and responses.

Display filter

```
udp.port == 53
```

I inspected the DNS information in the packet details and expanded the Queries section of the first packet.

Observation: The queried domain name was:

```
opensource.google.com
```

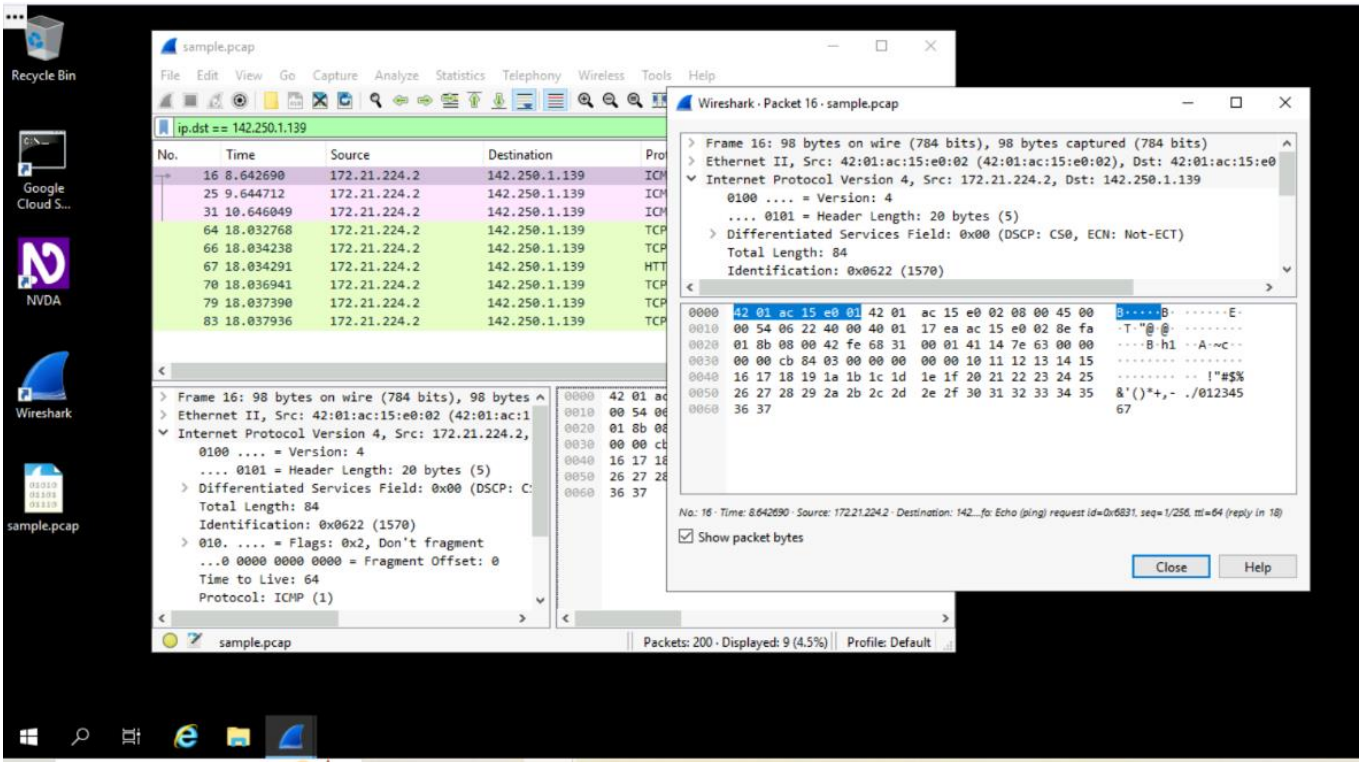
I then inspected the Answers section of the fourth packet in the filtered list.

Finding: The DNS response included the IP address `142.250.1.139` for the queried domain.

Screenshot 5 — DNS Query

[Insert Screenshot 5 here: Expand Domain Name System and Queries to show `opensource.google.com`.]

Screenshot 5 — DNS Response



Task 5: Explore TCP Packets

I used a TCP port filter to inspect packets associated with HTTP traffic.

Display filter

```
tcp.port == 80
```

I opened the first packet in the filtered list and examined the Frame and Internet Protocol Version 4 sections.

The packet details showed the following values:

Property	Observed value
Destination IP address	`169.254.169.254`
Time to Live (TTL)	`64`
Frame length	`54 bytes`
IPv4 header length	`20 bytes`

These values describe the selected packet in the provided sample capture. They should not be assumed to apply to every packet in a network.

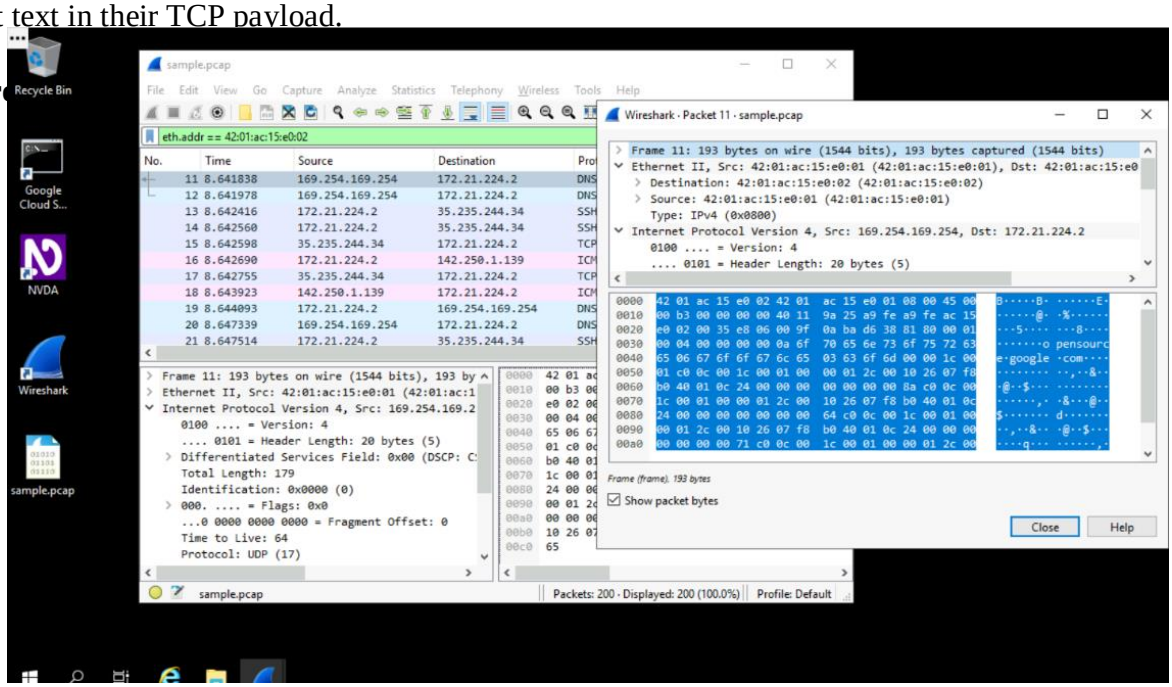
Search TCP payload for text

I then applied a filter to find TCP packets containing the text `curl`.

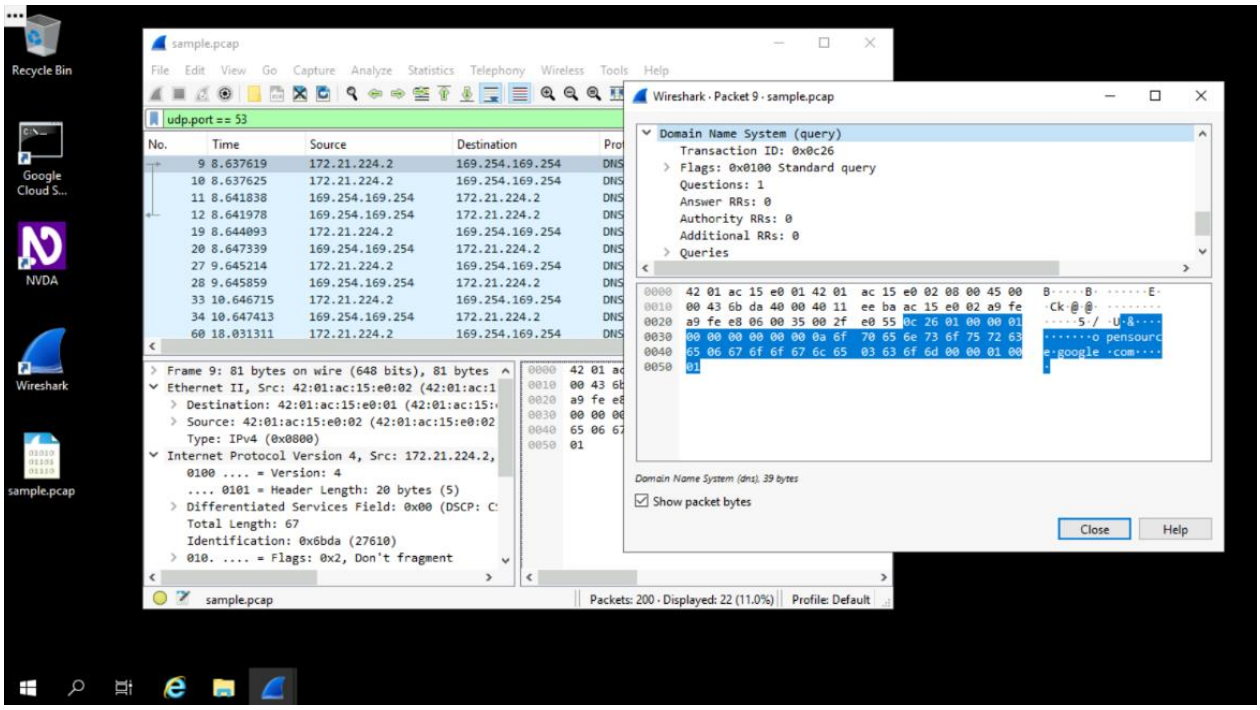
```
tcp contains "curl"
```

This filter searches TCP packet data for the specified text and helps identify packets containing that text in their TCP payload.

Scr



Screenshot 8 — TCP Payload Search



Key Findings

The lab demonstrated how Wireshark can be used to investigate captured network traffic.

- ICMP was used for the observed Echo (ping) request.
- The IP address filter helped isolate traffic associated with `142.250.1.139`.
- The inspected TCP packet used destination port `80`.
- MAC address filtering identified packets associated with a particular Ethernet address.
- DNS inspection revealed the query for `opensource.google.com` and its returned IP address.
- TCP filtering helped inspect HTTP-related traffic and search for specific payload text.

Key Learning

This activity helped me understand how to navigate Wireshark, inspect packet layers, and apply display filters to find relevant network traffic.

I also learned that packet analysis involves examining multiple layers of communication. IP addresses help identify network endpoints, MAC addresses identify interfaces at the Ethernet layer, ports help identify transport-layer services, and DNS records help explain how domain names are resolved.

Conclusion

Wireshark provides a practical way to inspect captured network traffic and investigate network communications. Understanding packet details and display filters is a useful foundation for further learning in network security, traffic analysis, and security operations center (SOC) investigations.